

FIREWALL SDN PROGRAMMABILE

Studente: Terracciano Gabriele

Matricola: DE9000165

Repository Github: <https://github.com/NCIs-unina/ncis-project-work-2026-terracciano-gabriele>

OBIETTIVO

L'obiettivo del progetto è realizzare un semplice **firewall in ambiente SDN (Software Defined Networking)**, in grado di controllare il traffico tra diversi host e di bloccare alcune comunicazioni in base a regole definite nel controller.

L'utilizzo di una rete SDN permette di separare il funzionamento dello switch dalle decisioni su come gestire il traffico e, dunque, permette di separare il **Control Plane**, responsabile delle decisioni sul traffico, dal **Data Plane**, responsabile dell'inoltro dei pacchetti.

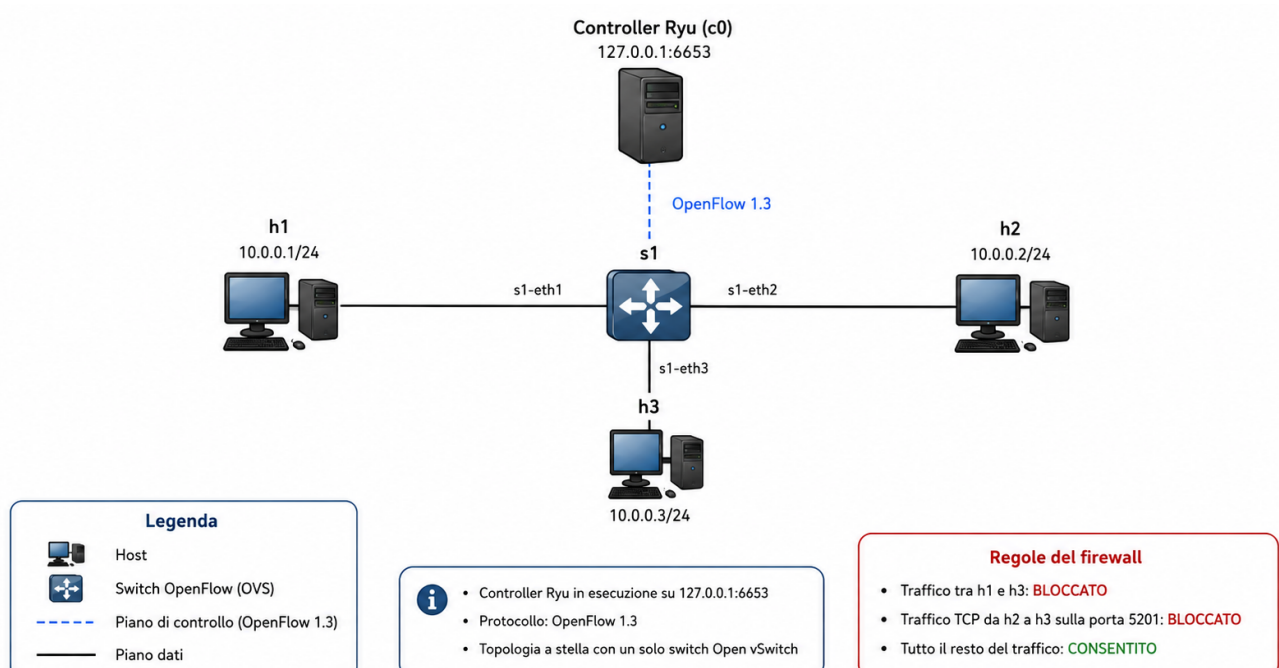
In questo caso è il **controller Ryu** a stabilire quali comunicazioni devono essere consentite o bloccate, installando direttamente le opportune regole nello switch. In questo modo è possibile modificare le politiche della rete senza cambiare la topologia.

Per realizzare il progetto sono stati utilizzati:

- **Mininet**, per creare ed emulare la rete e gli host;
- **Open vSwitch**, utilizzato come switch virtuale;
- **Ryu**, utilizzato come controller SDN;
- **OpenFlow 1.3**, utilizzato per la comunicazione tra il controller Ryu e lo switch.

Il progetto mostra quindi, in maniera semplice, come un controller SDN possa programmare uno switch e decidere quale traffico inoltrare e quale invece scartare.

TOPOLOGIA



LOGICA DI FUNZIONAMENTO

Il progetto, come detto, realizza un semplice firewall SDN tramite il controller **Ryu**, che programma lo switch **Open vSwitch** usando **OpenFlow 1.3**.

Il comportamento della rete è il seguente:

- il traffico **IPv4** tra **h1** e **h3** è **bloccato** in entrambe le direzioni. La prima politica del firewall impedisce la comunicazione tra gli host **h1** e **h3** in entrambe le direzioni. Questo significa che:

- **h1 non può comunicare con h3**
- **h3 non può comunicare con h1**

Il test di connettività conferma che la comunicazione tra questi due host viene bloccata, mentre le altre comunicazioni continuano a funzionare correttamente.

- il traffico **TCP** da **h2** verso **h3** sulla porta **5201** è **bloccato**. La seconda politica è più selettiva. In questo caso non viene bloccato tutto il traffico tra **h2** e **h3**, ma solo il traffico **TCP** destinato alla porta **5201**.

Questa porta viene usata da **iperf3**, che è stato utilizzato per verificare il funzionamento della regola. Infatti:

- **h2 può fare ping verso h3**
- **h2 non può aprire una connessione TCP verso h3 sulla porta 5201**

Questo mostra che il firewall non si limita a bloccare interamente due host, ma può anche filtrare un servizio specifico.

- tutto il resto del traffico è **consentito**.

In pratica, il controller installa nello switch delle **regole OpenFlow** che stabiliscono quali pacchetti devono essere inoltrati e quali invece devono essere scartati.

È importante precisare che le regole del firewall hanno una **priorità più alta** rispetto alle normali regole di inoltro dello switch.

Questo è particolarmente rilevante perché lo switch può avere:

- regole normali, usate per inoltrare i pacchetti;
- regole del firewall, usate per bloccare certi tipi di traffico.

Quando un pacchetto corrisponde a più regole, OpenFlow applica quella con **priorità maggiore**. Di conseguenza, se un traffico è vietato dal firewall, viene **bloccato prima** di poter essere inoltrato.

In questo modo il firewall riesce a controllare correttamente il comportamento della rete.

TEST

Per verificare il corretto funzionamento del firewall è stato realizzato un test automatico tramite il file `test_firewall.py`.

I test principali sono stati:

- verifica della comunicazione tra **h1 e h2**, che deve essere consentita;
- verifica della comunicazione tra **h2 e h3**, che deve essere consentita;
- verifica della comunicazione tra **h1 e h3**, che deve essere bloccata;
- verifica del traffico **TCP da h2 verso h3 sulla porta 5201**, che deve essere bloccato.

Il test automatico ha prodotto il seguente risultato:

```
=====
SDN FIREWALL TESTS
=====

[PASS] h1 -> h2 allowed
[PASS] h2 -> h3 allowed
[PASS] h1 -> h3 blocked
[PASS] TCP h2 -> h3:5201 blocked

=====
4/4 TESTS PASSED
=====
```

Questo conferma che tutte le regole definite nel controller vengono applicate correttamente.

ESECUZIONE

Per eseguire il progetto sono necessari due terminali. Nel primo terminale viene avviato il controller Ryu tramite:

```
source ~/ryu-env/bin/activate
ryu-manager controller/firewall.py
```

Nel secondo terminale viene prima pulito l'ambiente Mininet e successivamente avviata la topologia:

```
sudo mn -c
sudo python3 topology/network.py
```

Per eseguire direttamente i test automatici, mantenendo Ryu attivo, è possibile utilizzare:

```
sudo python3 tests/test_firewall.py
```